ChatGPT

Optimisations complémentaires et correctifs utiles pour ton pack de 85 tweaks

Résumé exécutif

Ton `tweaks.cpp` couvre déjà beaucoup de leviers (power plan, UI, USB power saving, RSS/Auto-tuning, désactivation de services/animations, etc.). Le meilleur "ROI" supplémentaire en 2026, sans casser Fortnite [1] ni l'anti-cheat, vient surtout de trois axes **manquants** ou **pas assez cadrés** :

- **Sécurité/scan disque** : exclusions Microsoft [2] Defender ciblées (gain de stutters IO) — mais à faire proprement et réversible. [3]
- **VBS/HVCI (Memory Integrity)** : option impactante sur certains PC compétitifs ; doit être testée A/B et désactivable proprement. [4]
- **Réseau "documenté" vs "mythes"** : ton PDF insiste de se limiter aux réglages Microsoft (autotuning, RSS) et d'éviter les tweaks registre réseau "miracles". Or ton code applique Nagle/DNS cache + reset IP/Winsock + ECN/timestamps. Il faut **séparer "réparation réseau"** de "perf gaming" et garder les tweaks réseau **mesurables et supportés**. [5]

En plus, il y a 2 "tweaks à risques" dans ton code qui doivent devenir **optionnels** (pas dans le preset par défaut) :

- **TDR registry** : Microsoft [2] indique explicitement que ces clés sont destinées au **test/débogage pilote**, et que les utilisateurs finaux ne doivent pas les manipuler. [6]
- **bcdedit timers (useplatformtick/disabledynamictick/etc.)** : pas de recommandation officielle "perf input lag" ; comportement variable selon hardware. À conserver uniquement comme module expérimental A/B, pas "Always-on". [7]

Analyse des lacunes par rapport à tes 85 tweaks

Ce que tu as déjà (fort) mais à "gater" (mode optionnel)

- **TdrDelay/TdrDdiDelay/TdrLevel** : Microsoft encadre ces clés pour le dev drivers, pas pour optimiser un PC gamer. Le gain "stabilité" existe parfois pour des workloads compute longs, mais en jeu on préfère régler la stabilité via drivers/OC, pas via TDR. [6]
- **Reset réseau** (`netsh int ip reset`, `netsh winsock reset`) : ce n'est pas un tweak performance, c'est une action "repair" qui peut casser VPN, IP statique, stack réseau. À déplacer dans un bouton "Réparer réseau" séparé avec avertissement + reboot. [8]
- **Nagle off / TcpAckFrequency / TcpDelAckTicks** : ce sont des tweaks historiques, mais ton propre PDF recommandait de ne pas faire de tweaks registre réseau non documentés (et de se limiter à autotuning/RSS). À rendre optionnel et mesuré (A/B). [5]
- **Vidage de standby list via NtSetSystemInformation (0x50)** : pas une commande supportée "perf gaming"; elle peut augmenter les lectures disque ensuite (cache détruit) => stutters. À garder en "troubleshooting mémoire", pas "performance". (Ton PDF insiste aussi sur l'importance des frametimes p95/p99 plutôt que le FPS moyen, ce qui va être pénalisé si tu détruis les caches). [9]

Ce qui manque vraiment (et utile)

1) Exclusions Defender ciblées + rollback. [3]
2) Toggle VBS/HVCI + vérification + reboot requis. [10]
3) Bloquer les **drivers via Windows Update** (pour éviter l'écrasement de ton driver GPU stable). [11]
4) Module "MPO toggle" (Multi-Plane Overlay) **conditionnel** si stutter/flicker, avec rollback officiel NVIDIA [12] . [13]
5) Tuning NIC **piloté par cmdlets** (RSS/RSC/LSO/Interrupt Moderation/EEE) avec découverte et reset. [14]
6) Protocole de mesure et outillage (PresentMon + DPC tool) déjà dans ton PDF ; mais il te manque des scripts d'automatisation basiques. [15]

Tweaks supplémentaires recommandés et "très utiles" (avec commandes et rollback)

Le tableau ci-dessous liste des ajouts à haute valeur, **documentés** ou supportés (Microsoft / vendors), avec **impact**, **risque**, **rollback**.

Tableau des ajouts prioritaires

Ajout	Pourquoi c'est utile	Comment l'appliquer (exemple)	Risque	Rollback
Exclusions Microsoft [2] Defender (process + paths)	Réduit scans temps réel sur fichiers jeux/ anti-cheat → moins de stutters IO	`Add-MpPreference -ExclusionPath "D:\Games\Fortnite"` et/ou `-ExclusionProcess`	Sécurité ↓ si trop large	`Remove-MpPrefere` exclusions [16]
Toggle Memory Integrity (HVCI) / VBS	Peut réduire overhead sur certaines configs compétitives ; doit être A/B	Via Windows Security ou registry/GP + reboot	Sécurité ↓ si OFF	Remettre ON + reboo
Bloquer drivers via Windows Update	Empêche WU de remplacer driver GPU stable (source fréquente de stutters)	Policy `ExcludeWUDriversInQualityUpdate=1`	Drivers non mis à jour via WU	remettre à 0 / désact [11]

Ajout	Pourquoi c'est utile	Comment l'appliquer (exemple)	Risque	Rollback
MPO toggle (NVIDIA [12])	Fix connu pour flicker/stutter desktop/jeux sur certains drivers	Appliquer `mpo_disable.reg` officiel + reboot	Peut changer comportement overlay	`mpo_restore.reg`
NIC tuning "latence": Interrupt Moderation / EEE	Peut réduire la latence réseau au prix d'un CPU plus haut (à tester)	`Set-NetAdapterAdvancedProperty ...`	CPU ↑, selon NIC	`Reset-NetAdapterAdvanc` [17]
RSC/RSS/LSO (NIC offloads)	RSC/LSO réduisent overhead CPU ; parfois la latence ↑ → A/B	`Set-NetAdapterRsc`, `Set-NetAdapterRss`, `Set-NetAdapterLso`	variable selon NIC/driver	remettre valeurs pré

Exclusions Defender (module prêt à intégrer)

Pourquoi : Defender peut scanner les exécutables/fichiers du jeu et provoquer de la contention disque CPU. Microsoft documente officiellement le principe et les cmdlets. [3]

PowerShell (Admin) – exemple prudent (à adapter aux chemins réels) :

```
# Dossiers
Add-MpPreference -ExclusionPath "D:\Games\Fortnite"
Add-MpPreference -ExclusionPath "C:\Program Files\Epic Games"

# Process (exemples, à valider sur ton install)
Add-MpPreference -ExclusionProcess "FortniteClient-Win64-Shipping.exe"
Add-MpPreference -ExclusionProcess "EpicGamesLauncher.exe"
Add-MpPreference -ExclusionProcess "EasyAntiCheat_EOS.exe"
```

> Bon sens : reste **ciblé** (paths + process exacts), pas d'exclusion globale. [3]

VBS/HVCI (Memory Integrity) : toggle propre + vérification

Microsoft explique comment activer via Windows Security et pourquoi. [10]

Vérifier l'état (PowerShell) :

```
Get-CimInstance -ClassName Win32_DeviceGuard -Namespace
root\Microsoft\Windows\DeviceGuard
```

Décision compétitive : - Si tu mesures une baisse de frametimes p95/p99 ou de latence avec HVCI OFF, tu peux le laisser OFF, en assumant le trade-off sécurité. [19]
- Si tu n'observes rien : laisse ON. [20]

Empêcher Windows Update d'installer des drivers (stabilité drivers GPU)

C'est un ajout **très utile** quand tu cherches une config stable. Microsoft documente la policy (GPO/MDM/registry) `ExcludeWUDriversInQualityUpdate`. [11]

Registry (Admin) :

```
reg add "HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" ^
 /v ExcludeWUDriversInQualityUpdate /t REG_DWORD /d 1 /f
```

Rollback :

```
reg add "HKLM\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate" ^
 /v ExcludeWUDriversInQualityUpdate /t REG_DWORD /d 0 /f
```

> Important : ça n'empêche pas l'update Windows, ça évite surtout l'update "drivers via WU". [21]

Réseau : remplace/complète tes tweaks par du tuning "supporté" et mesurable

Ton PDF recommandait clairement : rester sur des réglages Microsoft documentés (autotuning normal, RSS enabled) et éviter les tweaks "miracles" registre.

Ce que tu peux ajouter sans casser (et avec rollback clair)

RSS : mieux que netsh global, adapter-level (PowerShell)

Microsoft documente `Set-NetAdapterRss` et explique le bénéfice (distribution du trafic sur plusieurs cœurs). [22]

```
# Liste
Get-NetAdapterRss

# Activer sur l'interface (ex: "Ethernet")
Set-NetAdapterRss -Name "Ethernet" -Enabled $true
```

Tu peux aussi jouer sur les *RSS profiles* (choix de logique d'affinité CPU) ; Microsoft documente le paramètre `-Profile`. [23]

RSC : à tester (CPU ↓, latence parfois ↑)

Microsoft décrit RSC : coalescing de plusieurs paquets pour réduire overhead. Sur un PC compétitif, ça peut être bien si tu es CPU-bound, moins bien si tu cherches la latence minimale. **A/B obligatoire.** [24]

```
Get-NetAdapterRsc
Set-NetAdapterRsc -Name "Ethernet" -IPv4Enabled $true -IPv6Enabled $true
# ou pour test latence :
Set-NetAdapterRsc -Name "Ethernet" -IPv4Enabled $false -IPv6Enabled $false
```

Interrupt Moderation / EEE : tuning "latence" (adapter-specific)

- Microsoft documente comment modifier les propriétés avancées via `Set-NetAdapterAdvancedProperty`. [25]
- Intel documente que **désactiver Interrupt Moderation** peut réduire la latence mais augmente le CPU. [26]
- Tu dois d'abord lister les noms exacts disponibles sur TON NIC : `Get-NetAdapterAdvancedProperty`. [27]

```
# Découverte
Get-NetAdapterAdvancedProperty -Name "Ethernet" | Select DisplayName,
DisplayValue

# Test latence (si la propriété existe)
Set-NetAdapterAdvancedProperty -Name "Ethernet" -DisplayName "Interrupt
Moderation" -DisplayValue "Disabled"

# Reset si besoin
Reset-NetAdapterAdvancedProperty -Name "Ethernet" -DisplayName "Interrupt
Moderation"
```

Ce que je te conseille de retirer de "perf réseau" (à déplacer en "repair")

- `netsh int ip reset` + `netsh winsock reset` doivent être un module **réparation** (avec avertissement + reboot), pas un module perf. [8]
- `ecncapability=enabled` et `timestamps=disabled` : ce sont des toggles dont l'intérêt "gaming" n'est pas universel, et qui peuvent compliquer le diagnostic réseau. Garde-les derrière un switch "expérimental" + A/B. (Ton PDF recommande la prudence et la mesurabilité.)

Stabilité et stutters : deux tweaks "conditionnels" très utiles

MPO (Multi-Plane Overlay) : toggle officiel NVIDIA [12]

Si tu as : flicker, stutter desktop, glitch overlay (souvent après certains drivers), NVIDIA fournit un correctif officiel : `mpo_disable.reg` + reboot, et un `mpo_restore.reg` pour rollback. [13]

Pourquoi c'est intéressant : ce tweak est **réversible**, vendor-documenté, et souvent plus pertinent que des "tweaks registry random". [13]

TDR : à sortir du preset "Fortnite"

Ton code force TDR Delay à 60 sec et TdrLevel à recover. Or Microsoft précise que ces clés sont destinées au test/débogage driver, et que les utilisateurs finaux ne devraient pas les modifier. [6]

Recommandation : - Mets un module "TDR (debug uniquement)" OFF par défaut. - Si tu as des crashes GPU, la voie standard est : stabilité OC/UV, driver stable, vérifier erreurs WHEA/observateur d'événements, pas TDR. (Le TDR de 60s peut juste transformer un crash en freeze long.) [28]

Méthode A/B pour valider que "plus de tweaks = mieux"

Ton PDF insiste : ne pas juger sur FPS moyen, mais sur **frametime p95/p99** et latence.

Outils recommandés et pourquoi

- PresentMon : capture des métriques CPU/GPU/display frame durations et percentiles, et export CSV. [29]
- LatencyMon : mesure DPC/ISR, hard pagefaults, utile pour identifier un driver qui provoque spikes. [30]

Protocole simple

1) "Baseline" (aucun module expérimental).
2) Appliquer **un** nouveau module (ex : Defender exclusions).
3) Capturer 3 runs identiques (3–5 min) et comparer p95/p99 frametime.
4) Garder seulement ce qui améliore p95/p99 ou la latence de façon répétable. [9]

Automatisation PresentMon (exemple)

PresentMon documente un mode capture qui sort moyenne + percentiles 90/95/99. [9]

Tu peux intégrer un bouton "Bench quick" dans ton utilitaire qui lance une capture pendant X minutes et écrit un CSV/JSON.

Proposition très concrète : "pack de nouveaux modules" à ajouter à ton C++

Tu peux ajouter 5 modules supplémentaires (sans gonfler le nombre de hacks) :

- `applyDefenderExclusions()` → `Add-MpPreference` (paths/process, configurables). [16]
- `applyHVCI_VBS_Toggle(bool enable)` + `checkHVCI()` → via Win32_DeviceGuard et la doc HVCI. [19]
- `applyWUDriverExclusion(bool enable)` → `ExcludeWUDriversInQualityUpdate`. [31]
- `applyMPO(bool disable)` → wrapper du `.reg` MPO disable/restore, ou écriture du DWORD + reboot flag. [13]

- `applyNicLowLatencyProfile(adapterName)` → `Set-NetAdapterRss/Rsc` + `Set/Reset-NetAdapterAdvancedProperty` (Interrupt Moderation) uniquement si disponible. [32]

Et surtout : transforme certains de tes tweaks actuels en **"Experimental"** (désactivés par défaut) : - TDR keys [6]
- bcdedit timers (useplatformtick/disabledynamictick) [7]
- Nagle off / DNS cache TTL / ECN/timestamps

Si tu veux, colle-moi juste : - ton GPU (NVIDIA/AMD + modèle), - Ethernet ou Wi-Fi, - si tu joues en G-SYNC/FreeSync (oui/non), et je te propose un "preset" **Stable Competitive** vs **Experimental** (avec les toggles à activer et ceux à laisser off) en gardant le même style que ton `tweaks.cpp`.

---

[1] [4] [10] [19] [20] Enable virtualization-based protection of code integrity
https://learn.microsoft.com/en-us/windows/security/hardware-security/enable-virtualization-based-protection-of-code-integrity?utm_source=chatgpt.com

[2] [13] After updating to NVIDIA Game Ready Driver 461.09 or ...
https://nvidia.custhelp.com/app/answers/detail/a_id/5157/~/after-updating-to-nvidia-game-ready-driver-461.09-or-newer%2C-some-desktop-apps?utm_source=chatgpt.com

[3] Configure custom exclusions for Microsoft Defender Antivirus
https://learn.microsoft.com/en-us/defender-endpoint/configure-exclusions-microsoft-defender-antivirus?utm_source=chatgpt.com

[5] Problèmes connus de performances TCP/IP
https://learn.microsoft.com/fr-fr/troubleshoot/windows-server/networking/tcpip-performance-known-issues?utm_source=chatgpt.com

[6] Test et débogage TDR pendant le développement des ...
https://learn.microsoft.com/fr-fr/windows-hardware/drivers/display/tdr-registry-keys?utm_source=chatgpt.com

[7] BCDEdit /set - Windows drivers
https://learn.microsoft.com/en-us/windows-hardware/drivers/devtest/bcdedit--set?utm_source=chatgpt.com

[8] netsh interface
https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/netsh-interface?utm_source=chatgpt.com

[9] [15] PresentMon/README-CaptureApplication.md at main
https://github.com/GameTechDev/PresentMon/blob/main/README-CaptureApplication.md?utm_source=chatgpt.com

[11] [21] [31] Configure Windows Update client policies
https://learn.microsoft.com/en-us/windows/deployment/update/waas-configure-wufb?utm_source=chatgpt.com

[12] [23] Choosing a Network Adapter
https://learn.microsoft.com/en-us/windows-server/networking/technologies/network-subsystem/net-sub-choose-nic?utm_source=chatgpt.com

[14] [17] [25] [27] Set-NetAdapterAdvancedProperty (NetAdapter)
https://learn.microsoft.com/en-us/powershell/module/netadapter/set-netadapteradvancedproperty?view=windowsserver2025-ps&utm_source=chatgpt.com

[16] Add-MpPreference (Defender)
https://learn.microsoft.com/en-us/powershell/module/defender/add-mppreference?view=windowsserver2025-ps&utm_source=chatgpt.com

18 24 Set-NetAdapterRsc (NetAdapter)
https://learn.microsoft.com/fr-fr/powershell/module/netadapter/set-netadapterrsc?view=windowsserver2025-ps&utm_source=chatgpt.com

22 32 Set-NetAdapterRss (NetAdapter)
https://learn.microsoft.com/fr-fr/powershell/module/netadapter/set-netadapterrss?view=windowsserver2022-ps&utm_source=chatgpt.com

26 Interrupt Moderation - 1.1 | Intel® Ethernet 700/800 Series
https://edc.intel.com/content/www/id/id/design/products/ethernet/appnote-perf-tuning-guide-700-800-series-windows/%25E2%2580%258Binterru
moderation/https%3A%252F%252Fedc.intel.com%252Fcontent%252Fwww%252Fcn%252Fzh%252Fdesign%252Fproducts%252Fethernet%252Fappn
perf-tuning-guide-700-800-series-windows%252F%25E2%2580%258Binterrupt-moderation%252F/
https%3A%2F%2Fedc.intel.com%2Fcontent%2Fwww%2Fid%2Fid%2Fdesign%2Fproducts%2Fethernet%2Fappnote-perf-tuning-guide-700-800-series-
windows%2F%25E2%2580%258Binterrupt-
moderation%2Fhttps%3A%252F%252Fedc.intel.com%252Fcontent%252Fwww%252Fcn%252Fzh%252Fdesign%252Fproducts%252Fethernet%252Fa
perf-tuning-guide-700-800-series-windows%252F%25E2%2580%258Binterrupt-moderation%252F%2F?utm_source=chatgpt.com

28 WDDM Support for Timeout Detection and Recovery (TDR)
https://learn.microsoft.com/en-us/windows-hardware/drivers/display/timeout-detection-and-recovery?utm_source=chatgpt.com

29 GameTechDev/PresentMon
https://github.com/GameTechDev/PresentMon?utm_source=chatgpt.com

30 LatencyMon: suitability checker for real-time audio and ...
https://www.resplendence.com/latencymon?utm_source=chatgpt.com